LAPORAN PRAKTIKUM PEMROGRAMAN WEB
MODUL 8: HMVC, AUTHORIZATION, DAN AJAX PADA LARAVEL

Disusun Oleh:

Nama: Andhika Yuridho

NIM: 707012500059

Kelas: D4SIKC-49-04

Mata Kuliah: Pemrograman Web

UNIVERSITAS TELKOM

FAKULTAS ILMU TERAPAN

JURUSAN S1 TERAPAN SISTEM INFORMASI KOTA CERDAS

TAHUN AJARAN 2026

1. Tujuan

Tugas akhir Modul 8 yang saya kerjakan adalah soal nomor 1 pada bagian Tugas Praktikum, yaitu aplikasi SiPustaka untuk perpustakaan kampus. Aplikasi ini dibangun dari project Laravel baru dengan arsitektur HMVC. Tujuan pengerjaannya adalah sebagai berikut:

1. Memecah aplikasi menjadi modul Buku, Peminjaman, dan UserManagement memakai package nwidart/laravel-modules.

2. Membuat fitur register, login, dan logout secara manual di luar modul, serta menambahkan kolom role admin dan user pada tabel users.

3. Menerapkan middleware role dan mendaftarkan aliasnya di bootstrap/app.php.

4. Membangun CRUD buku secara synchronous yang hanya dapat diubah admin, lengkap dengan pencarian dan pagination.

5. Membangun CRUD peminjaman berbasis AJAX untuk admin dan user, termasuk unggah berkas bukti.

6. Mengelola akun pengguna pada modul UserManagement dengan batasan admin tidak dapat menghapus atau menurunkan role akunnya sendiri.

7. Menampilkan pesan validasi di bawah input dan pesan berhasil setelah data ditambah, diubah, atau dihapus.

2. Alat dan Bahan

Praktikum ini dikerjakan pada satu laptop Lenovo 83DV dengan prosesor Intel Core i7-13650HX (14 core, 20 thread), RAM 12 GB, dan Windows 11 Pro versi 10.0.26100. Perangkat lunak yang dipakai adalah Laragon sebagai server lokal dengan MySQL 8.4.3 dan phpMyAdmin 5.2.3; PHP 8.3.30 (ZTS); Composer 2.10.3; Laravel Framework 13.40.0; nwidart/laravel-modules 13.3.0; Bootstrap 5.3.0, Bootstrap Icons 1.11.3, dan jQuery 3.6.0 yang dimuat melalui CDN; Visual Studio Code; Google Chrome; serta Git.

Aplikasi memakai database MySQL db_sipustaka. Server pengembangan dijalankan dengan php artisan serve pada alamat http://127.0.0.1:8126. Berkas uji unggahan berupa satu gambar PNG berukuran 1,6 KB dan satu arsip ZIP untuk menguji penolakan format.

3. Langkah-Langkah Praktikum

Pengerjaan dimulai dari skeleton Laravel yang sudah ada di folder praktikum8-ta, lalu dilanjutkan dengan pemasangan package modul, pembuatan authentication, middleware role,

tiga modul, dan pengujian di browser. Seluruh perintah dijalankan dari terminal di dalam folder project.

3.1 Menyiapkan Project SiPustaka dan Modul HMVC

Folder praktikum8-ta sudah berisi skeleton Laravel 13.40.0, jadi saya tidak menjalankan composer create-project lagi. Nama aplikasi pada file .env diisi SiPustaka, APP_LOCALE diubah menjadi id, dan koneksi database diarahkan ke MySQL dengan DB_DATABASE=db_sipustaka. Database dibuat melalui phpMyAdmin, lalu package modul dipasang dengan composer config allow-plugins.wikimedia/composer-merge-plugin true dan composer require nwidart/laravel-modules.

Setelah itu saya mempublikasikan konfigurasi package dengan php artisan vendor:publish --provider="Nwidart\Modules\LaravelModulesServiceProvider", menambahkan bagian merge-plugin pada composer.json, lalu membuat tiga modul sekaligus memakai php artisan module:make Buku Peminjaman UserManagement. Perintah composer dump-autoload dijalankan setelah composer.json berubah, dan php artisan module:list menunjukkan ketiga modul berstatus Enabled.

Pada percobaan pertama, composer dump-autoload berhenti dengan pesan Class "Modules\Buku\Providers\BukuServiceProvider" not found. Penyebabnya, bagian merge-plugin belum ada di composer.json sehingga file composer.json milik setiap modul belum ikut dibaca. Setelah bagian itu ditambahkan, autoload berhasil dibuat dan ketiga modul dapat dimuat.

Modul praktikum ini memakai Laravel 12 dan nwidart/laravel-modules versi 12, sedangkan Composer di laptop ini membuat Laravel 13.40.0 sehingga package modul yang terpasang adalah versi 13.3.0. Struktur modul dan perintah artisan yang dipakai sama, hanya penulisan model User mengikuti gaya Laravel 13 yang memakai atribut PHP untuk fillable dan hidden.

3.2 Membuat Role dan Authentication di Luar HMVC

Kolom role ditambahkan lewat migration baru pada tabel users. Nilainya berupa string dengan default user, dan kolom diletakkan setelah email.

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::table('users', function (Blueprint $table) {
            $table->string('role')->default('user')->after('email');
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::table('users', function (Blueprint $table) {
            $table->dropColumn('role');
        });
    }
};

```

Daftar role yang sah disimpan pada enum UserRole. Method label() mengembalikan tulisan Admin atau User untuk ditampilkan pada badge dan navbar.

```php
<?php

namespace App\Enums;

enum UserRole: string
{
    case Admin = 'admin';
    case User = 'user';

    /**
     * Get the human readable label of the role.
     */
    public function label(): string
    {
        return match ($this) {
            self::Admin => 'Admin',
            self::User => 'User',
        };
    }
}

```

Model User memakai cast role ke enum tersebut, ditambah method isAdmin() dan hasRole() yang dipakai oleh view dan middleware. UserFactory mendapat state admin(), sedangkan DatabaseSeeder membuat dua akun awal, yaitu admin@example.com dan user@example.com dengan password password.

Validasi login dan register dipisahkan ke Form Request. RegisterRequest memakai aturan unique pada email dan confirmed pada password, dan field role tidak ada di dalamnya sehingga akun hasil register selalu menjadi user. RegisterController mengisi role User secara eksplisit, sedangkan LoginController memakai RateLimiter untuk membatasi lima percobaan gagal per email dan alamat IP, lalu memanggil session regenerate setelah login berhasil.

Route authentication ditulis di routes/web.php. Halaman login dan register berada di dalam middleware guest, sedangkan dashboard dan logout berada di dalam middleware auth. Route milik modul tidak ditulis di file ini karena setiap modul mendaftarkan route-nya sendiri.

3.3 Membuat Middleware Role

Middleware EnsureUserHasRole menerima daftar role sebagai parameter. Jika pengguna belum login atau role-nya tidak termasuk yang diizinkan, request dihentikan dengan abort(403) dan pesan berbahasa Indonesia.

```php
<?php

namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;
use Symfony\Component\HttpFoundation\Response;

class EnsureUserHasRole
{
    /**
     * Ensure the authenticated user has one of the given roles.
     *
     * Usage: ->middleware('role:admin') or ->middleware('role:admin,user')
     *
     * @param  Closure(Request): (Response)  $next
     */
    public function handle(Request $request, Closure $next, string ...$roles): Response
    {
        $user = $request->user();

        if ($user === null || ! $user->hasRole(...$roles)) {
            abort(403, 'Anda tidak memiliki hak akses ke halaman ini.');
        }

        return $next($request);
    }
}

```

Middleware didaftarkan dengan alias role di bootstrap/app.php. Saya juga mengatur redirectGuestsTo ke route login dan redirectUsersTo ke route dashboard supaya tamu dan pengguna yang sudah login diarahkan ke halaman yang tepat.

```php
<?php

use App\Http\Middleware\EnsureUserHasRole;
use Illuminate\Foundation\Application;
use Illuminate\Foundation\Configuration\Exceptions;
use Illuminate\Foundation\Configuration\Middleware;
use Illuminate\Http\Request;

return Application::configure(basePath: dirname(__DIR__))
    ->withRouting(
        web: __DIR__.'/../routes/web.php',
        commands: __DIR__.'/../routes/console.php',
        health: '/up',
    )
    ->withMiddleware(function (Middleware $middleware): void {
        $middleware->alias([
            'role' => EnsureUserHasRole::class,
        ]);

        $middleware->redirectGuestsTo(fn () => route('login'));
        $middleware->redirectUsersTo(fn () => route('dashboard'));
    })
    ->withExceptions(function (Exceptions $exceptions): void {
        $exceptions->shouldRenderJsonWhen(
            fn (Request $request) => $request->is('api/*') || $request->expectsJson(),
        );
    })->create();

```

Urutan middleware pada route selalu auth lebih dulu, baru role. Dengan urutan itu tamu diarahkan ke halaman login, sedangkan pengguna yang sudah login tetapi role-nya tidak sesuai mendapat halaman 403.

3.4 Membuat Modul Buku (CRUD Synchronous)

Modul Buku menyimpan lima kolom, yaitu judul, penulis, penerbit, tahun_terbit, dan stok. Tahun terbit memakai unsignedSmallInteger, sedangkan stok memakai unsignedInteger dengan nilai awal 0. Model Buku mengikuti kolom tersebut dan tidak memakai relasi karena data peminjaman menyimpan judul buku sebagai teks.

Validasi tambah dan ubah buku ditulis pada StoreBukuRequest dan UpdateBukuRequest. Tahun terbit dibatasi antara 1900 dan tahun berjalan, sedangkan stok tidak boleh negatif. Pesan error ditulis dalam bahasa Indonesia agar langsung muncul di bawah input yang salah.

BukuController memakai pencarian sederhana dengan LIKE pada judul, penulis, dan penerbit, lalu mengurutkan hasil berdasarkan judul dan menampilkan lima baris per halaman. Method withQueryString() menjaga kata kunci tetap ada saat berpindah halaman.

```php
    /**
     * Display a paginated and searchable list of books.
     */
    public function index(Request $request): View
    {
        $search = $request->string('q')->trim()->toString();

        $bukus = Buku::query()
            ->when($search !== '', function ($query) use ($search) {
                $query->where(function ($query) use ($search) {
                    $query->where('judul', 'like', '%'.$search.'%')
                        ->orWhere('penulis', 'like', '%'.$search.'%')
                        ->orWhere('penerbit', 'like', '%'.$search.'%');
                });
            })
            ->orderBy('judul')
            ->paginate(5)
            ->withQueryString();

        return view('buku::index', [
            'bukus' => $bukus,
            'search' => $search,
        ]);
    }
```

Route modul dipisah menjadi dua kelompok. Semua pengguna yang sudah login boleh membuka daftar buku, sedangkan tambah, ubah, dan hapus hanya untuk admin.

```php
<?php

use Illuminate\Support\Facades\Route;
use Modules\Buku\Http\Controllers\BukuController;

Route::middleware(['auth', 'role:admin,user'])->group(function () {
    Route::get('bukus', [BukuController::class, 'index'])->name('buku.index');
});

Route::middleware(['auth', 'role:admin'])->group(function () {
    Route::get('bukus/create', [BukuController::class, 'create'])->name('buku.create');
    Route::post('bukus', [BukuController::class, 'store'])->name('buku.store');
    Route::get('bukus/{buku}/edit', [BukuController::class, 'edit'])->name('buku.edit');
    Route::put('bukus/{buku}', [BukuController::class, 'update'])->name('buku.update');
    Route::delete('bukus/{buku}', [BukuController::class, 'destroy'])-
>name('buku.destroy');
});

```

Tampilan daftar buku memakai tabel Bootstrap. Kolom aksi hanya muncul ketika auth()->user()->isAdmin() bernilai benar, sehingga user melihat daftar tanpa tombol tambah, edit, atau hapus. Form tambah dan edit memakai satu partial yang sama, dengan old() untuk mengembalikan isian setelah validasi gagal.

3.5 Membuat Modul Peminjaman (CRUD AJAX dan Upload Bukti)

Modul Peminjaman menyimpan nama_peminjam, judul_buku, tanggal_pinjam, tanggal_kembali, status, dan bukti. Kolom status hanya berisi Dipinjam atau Dikembalikan, sedangkan bukti menyimpan path berkas dan boleh kosong.

Pada percobaan pertama, Eloquent membentuk nama tabel peminjamens dari nama model Peminjaman karena aturan jamak bahasa Inggris diterapkan pada kata Indonesia. Dashboard sempat menampilkan error 500 dengan pesan Table db_sipustaka.peminjamen doesn't exist. Saya menambahkan properti $table = 'peminjamans' pada model agar nama tabelnya sesuai migration.

StorePeminjamanRequest dan UpdatePeminjamanRequest memvalidasi tanggal kembali agar tidak mendahului tanggal pinjam, status yang hanya boleh dua nilai, serta berkas bukti dengan format jpg, jpeg, png, atau pdf dan ukuran maksimal 2 MB.

```php
<?php

namespace Modules\Peminjaman\Http\Requests;

use Illuminate\Foundation\Http\FormRequest;

class StorePeminjamanRequest extends FormRequest
{
    /**
     * Determine if the user is authorized to make this request.
     */
    public function authorize(): bool
    {
        return $this->user() !== null;
    }

    /**
     * Get the validation rules that apply to the request.
     *
     * @return array<string, array<int, mixed>>
     */
    public function rules(): array
    {
        return [
            'nama_peminjam' => ['required', 'string', 'max:255'],
            'judul_buku' => ['required', 'string', 'max:255'],
            'tanggal_pinjam' => ['required', 'date'],
            'tanggal_kembali' => ['required', 'date', 'after_or_equal:tanggal_pinjam'],
            'status' => ['required', 'in:Dipinjam,Dikembalikan'],
            'bukti' => ['nullable', 'file', 'mimes:jpg,jpeg,png,pdf', 'max:2048'],
        ];
    }

    /**
     * Get the error messages for the defined validation rules.
     *
     * @return array<string, string>
     */
    public function messages(): array
    {
        return [
            'nama_peminjam.required' => 'Nama peminjam wajib diisi.',
            'judul_buku.required' => 'Judul buku wajib diisi.',
            'tanggal_pinjam.required' => 'Tanggal pinjam wajib diisi.',
            'tanggal_pinjam.date' => 'Tanggal pinjam tidak valid.',
            'tanggal_kembali.required' => 'Tanggal kembali wajib diisi.',
            'tanggal_kembali.after_or_equal' => 'Tanggal kembali tidak boleh sebelum
tanggal pinjam.',
            'status.required' => 'Status wajib dipilih.',
            'status.in' => 'Status harus Dipinjam atau Dikembalikan.',
            'bukti.mimes' => 'Bukti harus berupa berkas jpg, jpeg, png, atau pdf.',
            'bukti.max' => 'Ukuran bukti maksimal 2 MB.',
        ];
    }
}

```

PeminjamanController mengembalikan JSON, bukan view. Method data() mengirim seluruh baris untuk tabel, method show() mengirim satu baris untuk modal edit, dan method store(),

update(), serta destroy() mengembalikan pesan. Berkas baru disimpan ke disk public di folder bukti_peminjaman. Saat edit, berkas lama dihapus lebih dulu jika pengguna memilih berkas pengganti. Saat hapus, berkas ikut dihapus sebelum barisnya dihapus.

```php
    /**
     * Store a new loan.
     */
    public function store(StorePeminjamanRequest $request): JsonResponse
    {
        $data = $request->safe()->except('bukti');

        if ($request->hasFile('bukti')) {
            $data['bukti'] = $request->file('bukti')->store('bukti_peminjaman', 'public');

        }

        $peminjaman = Peminjaman::create($data);

        return response()->json([
            'message' => 'Peminjaman berhasil ditambahkan.',
            'id' => $peminjaman->id,
        ], 201);
    }

    /**
     * Update the given loan.
     */
    public function update(UpdatePeminjamanRequest $request, Peminjaman $peminjaman):
JsonResponse
    {
        $data = $request->safe()->except('bukti');

        if ($request->hasFile('bukti')) {
            if ($peminjaman->bukti) {
                Storage::disk('public')->delete($peminjaman->bukti);
            }

            $data['bukti'] = $request->file('bukti')->store('bukti_peminjaman', 'public');

        }

        $peminjaman->update($data);

        return response()->json(['message' => 'Peminjaman berhasil diperbarui.']);
    }

    /**
     * Remove the given loan and its uploaded file.
     */
    public function destroy(Peminjaman $peminjaman): JsonResponse
    {
        if ($peminjaman->bukti) {
            Storage::disk('public')->delete($peminjaman->bukti);
        }

        $peminjaman->delete();

        return response()->json(['message' => 'Peminjaman berhasil dihapus.']);
    }

```

Halaman peminjaman memuat tabel kosong, satu modal form, dan modal konfirmasi hapus. Data diambil dengan $.getJSON, lalu baris tabel disusun dari respons JSON. Semua teks dari

server dimasukkan melalui escapeHtml() agar karakter seperti <script> tidak dijalankan browser.

Form dikirim dengan FormData dan pengaturan processData: false serta contentType: false, sehingga berkas ikut terkirim sebagai multipart/form-data. Token CSRF diambil dari meta csrf-token dan dipasang pada $.ajaxSetup. Saat mengubah data, FormData diberi _method=PUT karena jQuery mengirim POST. Jika server membalas 422, pesan errors ditampilkan di bawah input yang bersangkutan.

```
form.addEventListener('submit', function (event) {
    event.preventDefault();
    clearErrors();

    const data = new FormData(form);
    const button = document.getElementById('btnSimpan');
    button.disabled = true;

    if (editingId) {
        data.append('_method', 'PUT');
    }

    const url = editingId ? urls.base + '/' + editingId : urls.store;

    $.ajax({
        url: url,
        method: 'POST',
        data: data,
        processData: false,
        contentType: false,
    })
        .done(function (response) {
            modal.hide();
            showAlert(response.message, 'success');
            loadData();
        })
        .fail(function (xhr) {
            if (xhr.status === 422) {
                showErrors(xhr.responseJSON.errors || {});
            } else {
                showAlert('Terjadi kesalahan saat menyimpan data.', 'danger');

            }
        })
        .always(function () {
            button.disabled = false;
        });
});

```

Route peminjaman berada di dalam middleware auth dan role:admin,user. Route peminjamans/data diletakkan sebelum route yang memakai parameter {peminjaman} agar kata data tidak dianggap sebagai id.

3.6 Membuat Modul UserManagement

Modul UserManagement memakai model App\Models\User milik aplikasi utama, jadi modul ini tidak memiliki model sendiri. StoreUserRequest mewajibkan nama, email unik, role, dan password dengan konfirmasi. UpdateUserRequest memakai aturan unique yang mengabaikan email milik user yang sedang diedit, dan password boleh kosong jika tidak ingin diubah.

UserManagementController menyediakan daftar, tambah, edit, dan hapus akun. Ada dua penjagaan di dalam controller. Saat update, admin yang mengubah akunnya sendiri tidak boleh memilih role selain admin. Saat hapus, admin tidak boleh menghapus akunnya sendiri. Keduanya mengembalikan pesan error tanpa mengubah data.

```php
    /**
     * Update the given user.
     */
    public function update(UpdateUserRequest $request, User $user): RedirectResponse
    {
        $data = $request->validated();

        if ($request->user()->is($user) && $data['role'] !== UserRole::Admin->value) {
            return back()
                ->withInput()
                ->withErrors(['role' => 'Anda tidak dapat menurunkan role akun sendiri.']);
        }

        if (empty($data['password'])) {
            unset($data['password']);
        }

        $user->update($data);

        return redirect()->route('admin.users.index')->with('success', 'User berhasil diperbarui.');
    }

    /**
     * Remove the given user.
     */
    public function destroy(Request $request, User $user): RedirectResponse
    {
        if ($request->user()->is($user)) {
            return redirect()->route('admin.users.index')->with('error', 'Anda tidak dapat menghapus akun sendiri.');
        }

        $user->delete();

        return redirect()->route('admin.users.index')->with('success', 'User berhasil dihapus.');
    }
}
```

Seluruh route modul memakai prefix admin dan middleware role:admin, sehingga user biasa yang membuka /admin/users langsung mendapat halaman 403.

3.7 Menyusun Layout dan Aset Bersama

Bagian head dan script dipisah menjadi partial. Partial head memuat meta csrf-token, Bootstrap CSS, Bootstrap Icons, dan file public/css/theme.css. Partial scripts memuat jQuery, bundle Bootstrap, modal konfirmasi hapus, tombol tampilkan password, dan penutup otomatis untuk pesan sukses setelah empat detik.

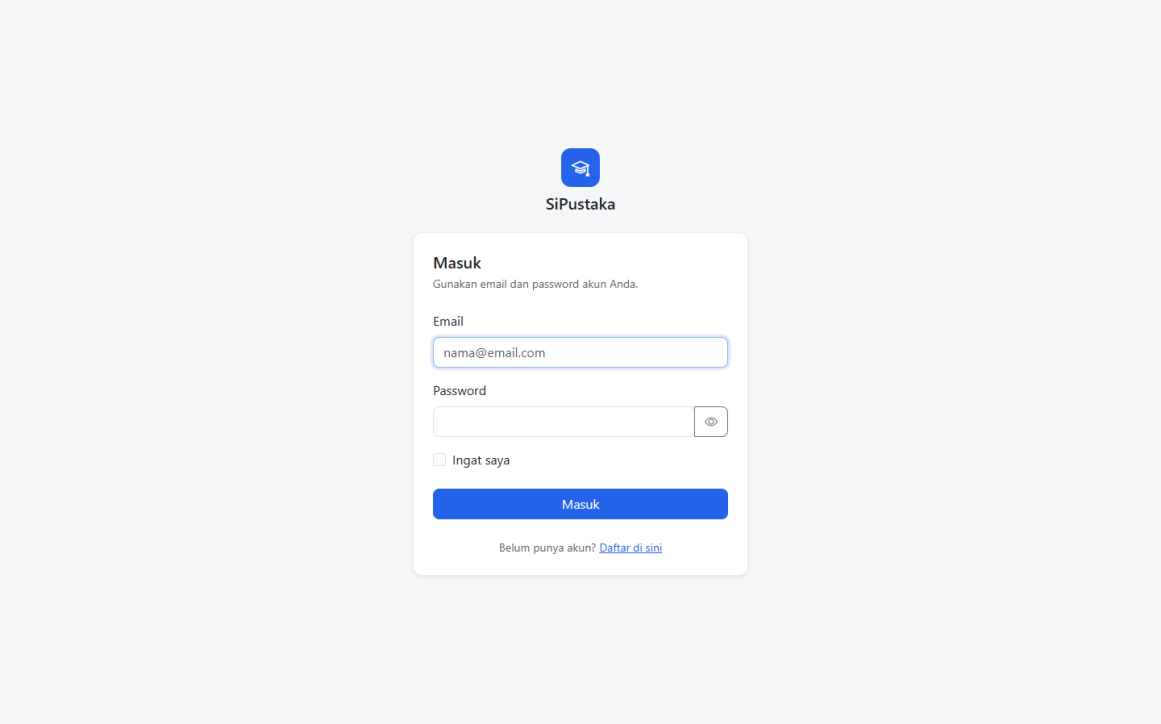
Ada dua layout. layouts/app.blade.php dipakai halaman yang memerlukan login dan memuat navbar dengan menu Dashboard, Buku, Peminjaman, serta Kelola User yang hanya tampil

untuk admin. layouts/guest.blade.php dipakai halaman login dan register dengan kartu di tengah layar. File theme.css hanya berisi penyesuaian warna, sudut kartu, dan tampilan tabel di atas Bootstrap.

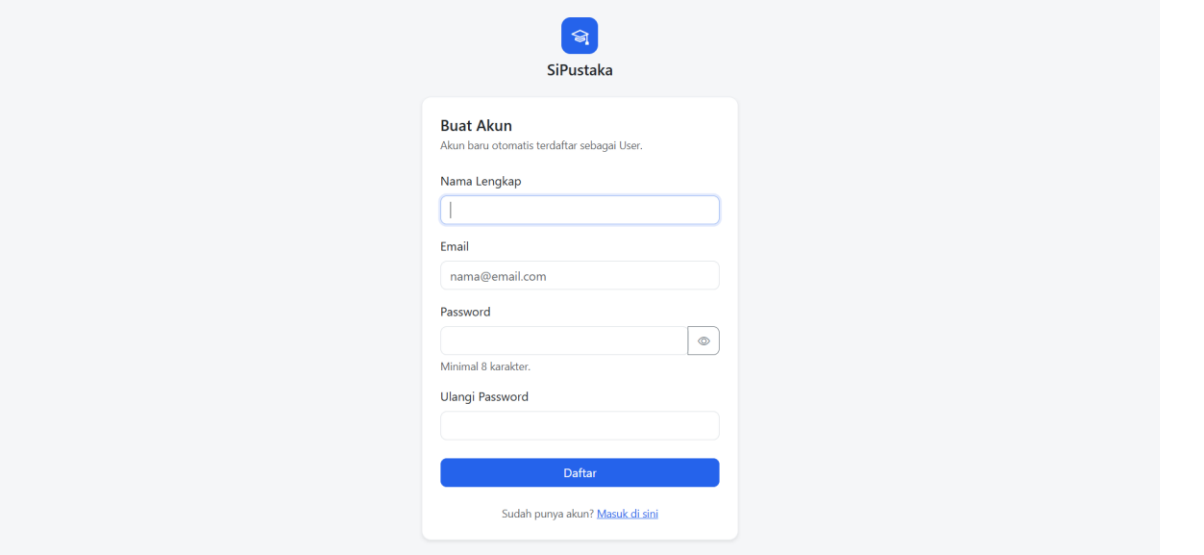
Fungsi confirmDelete() mengembalikan Promise bernilai true ketika tombol Ya, hapus ditekan. Fungsi ini dipakai form biasa melalui atribut data-confirm dan juga dipakai tombol hapus pada tabel AJAX.

4. Hasil dan Dokumentasi Program

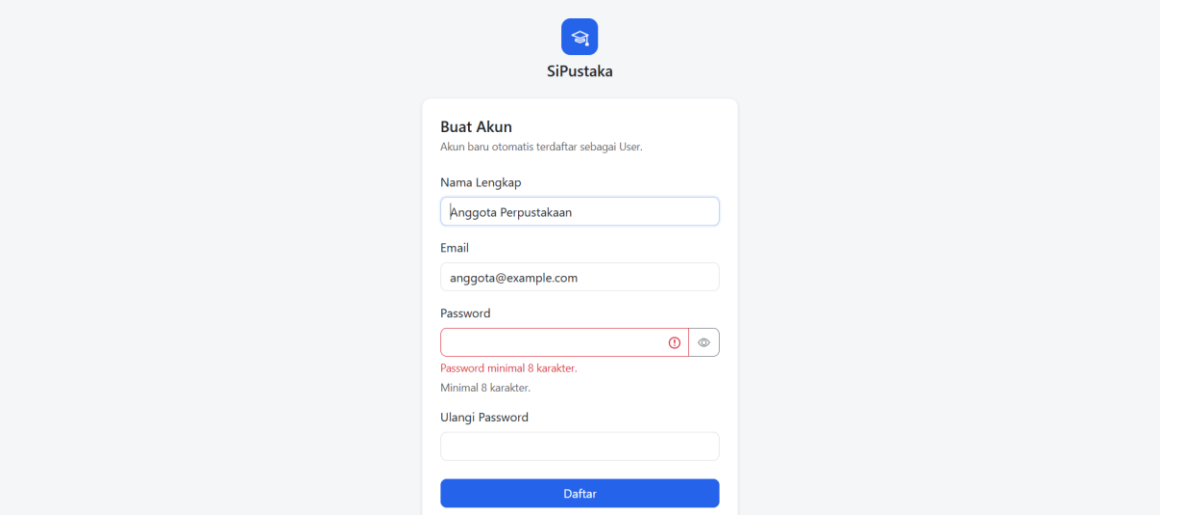
Seluruh halaman diuji pada browser dengan dua akun, yaitu Administrator sebagai admin dan Anggota Perpustakaan sebagai user. Akun user kedua dibuat melalui halaman register.



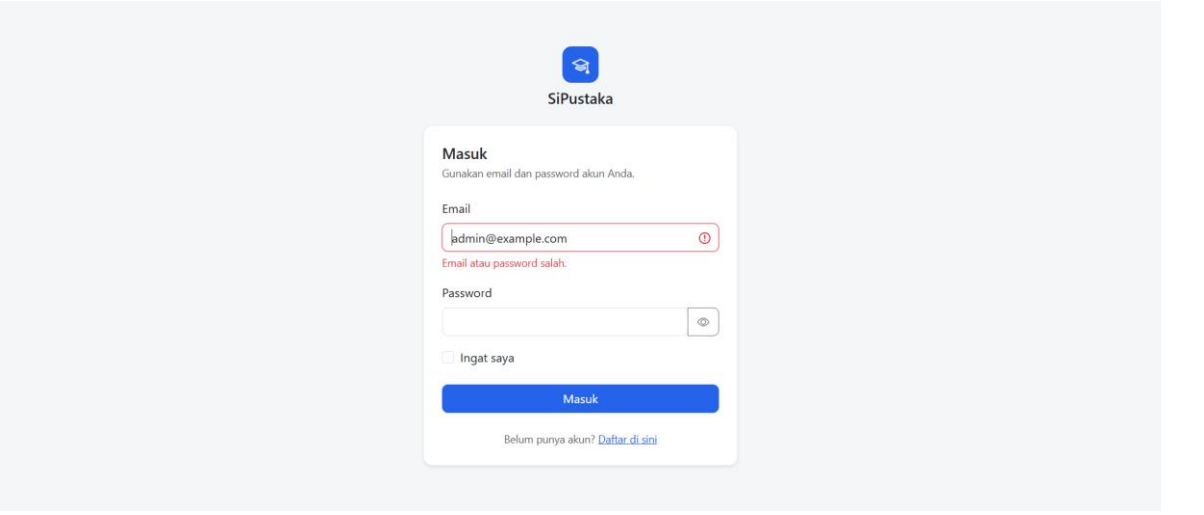
Gambar 1. Halaman login SiPustaka



Gambar 2. Halaman register dengan keterangan bahwa akun baru berperan User

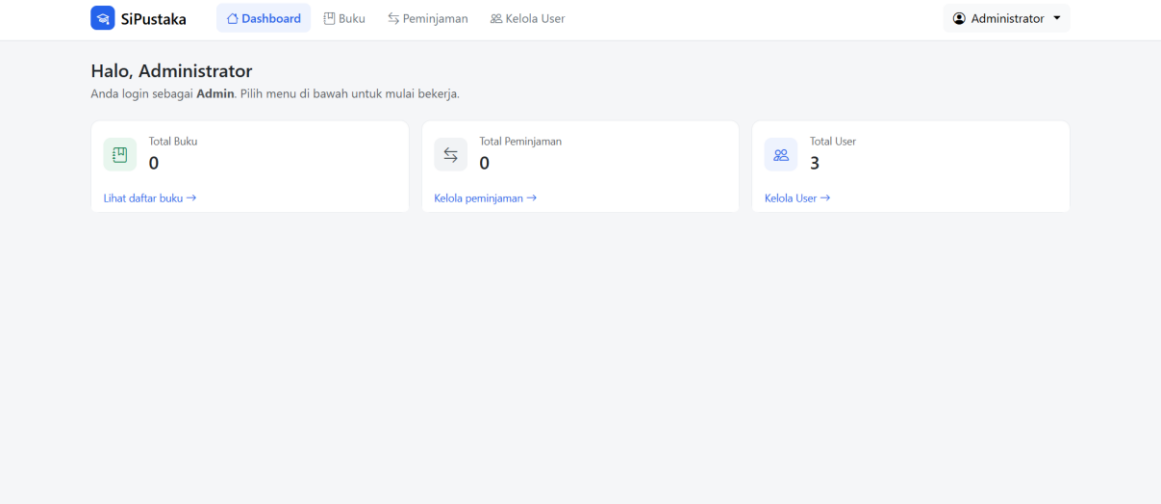


Gambar 3. Pesan validasi ketika password kurang dari 8 karakter

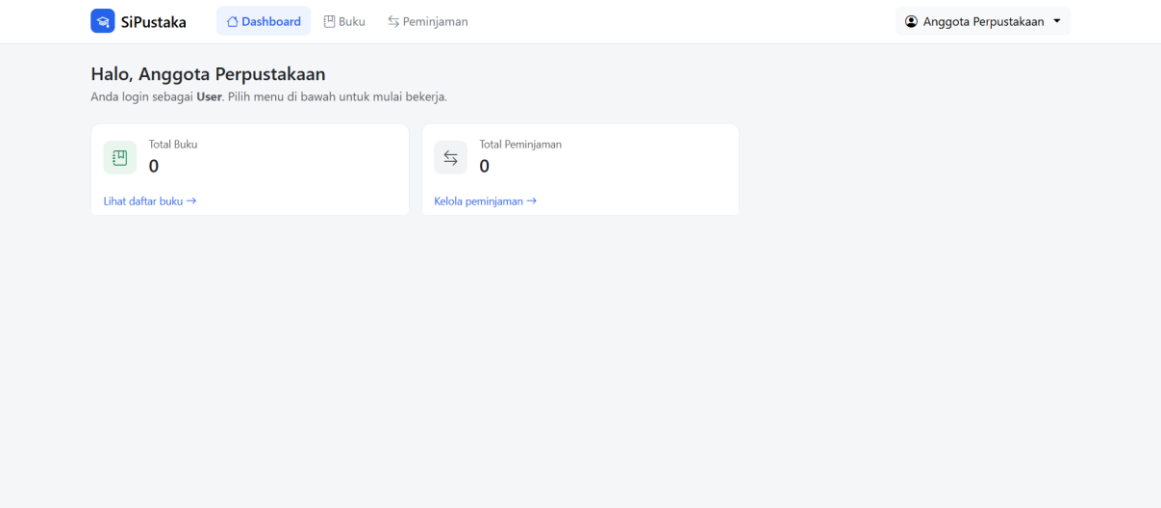


Gambar 4. Pesan error ketika password yang dimasukkan salah

Setelah login, dashboard menampilkan jumlah buku, jumlah peminjaman, dan jumlah user. Kartu Total User hanya muncul untuk admin, dan menu Kelola User juga hanya tampil pada navbar admin.

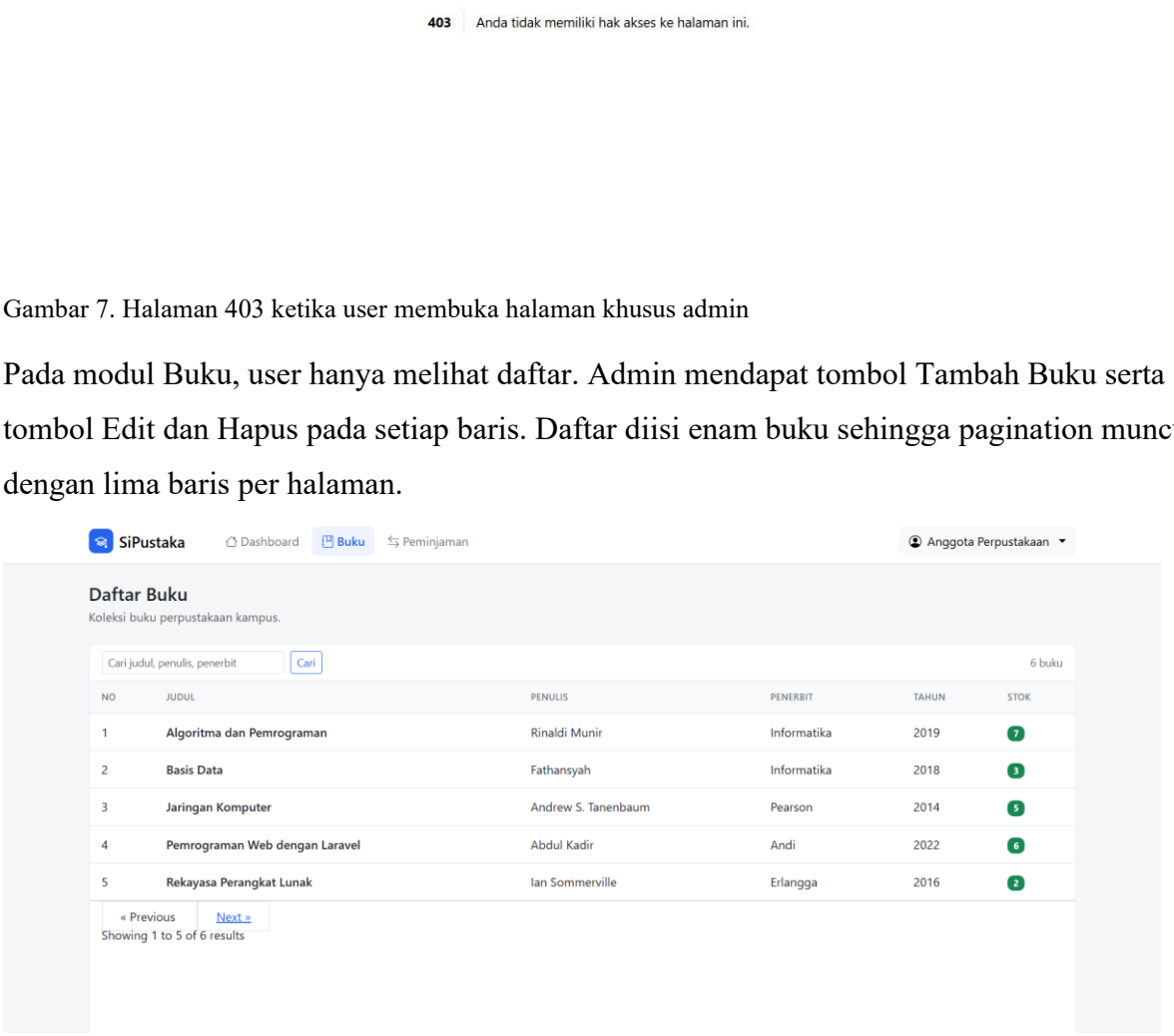


Gambar 5. Dashboard admin dengan tiga kartu ringkasan



Gambar 6. Dashboard user tanpa kartu Total User dan tanpa menu Kelola User

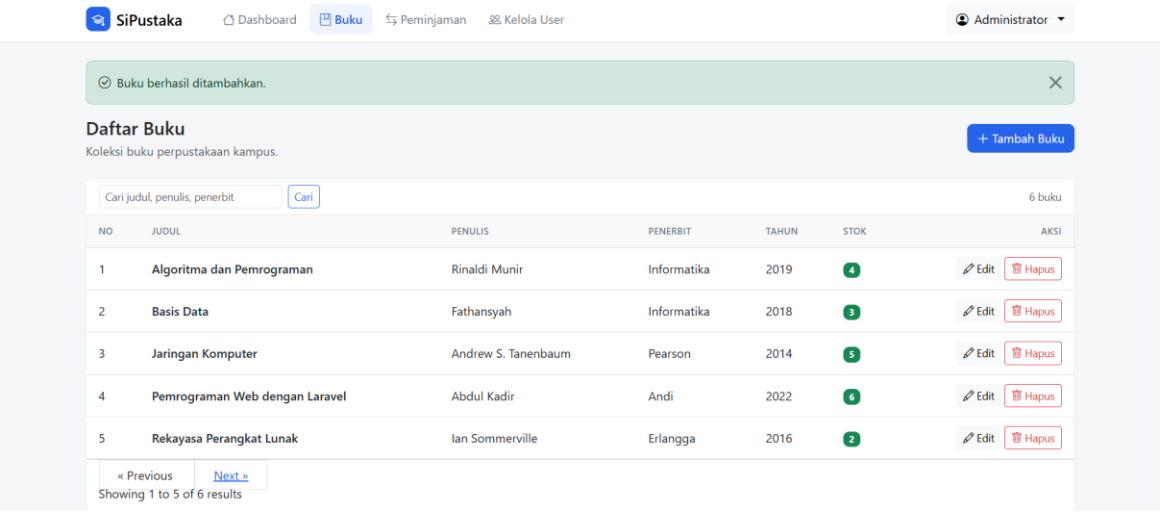
Pembuktian authorization dilakukan dengan membuka /admin/users memakai akun user. Middleware role menghentikan request dan menampilkan halaman 403 dengan pesan yang saya tulis sendiri.



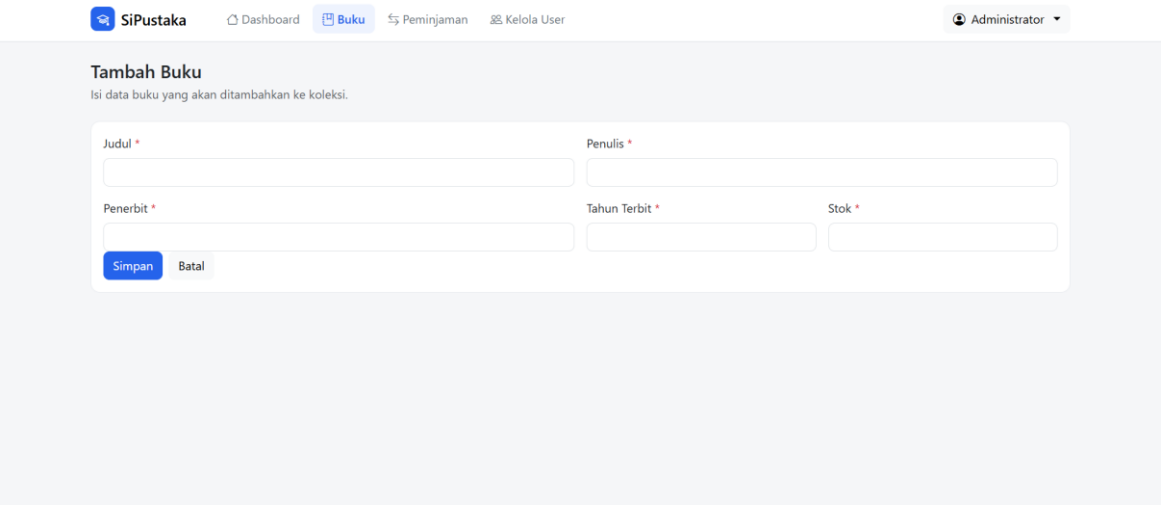
Gambar 7. Halaman 403 ketika user membuka halaman khusus admin

Pada modul Buku, user hanya melihat daftar. Admin mendapat tombol Tambah Buku serta tombol Edit dan Hapus pada setiap baris. Daftar diisi enam buku sehingga pagination muncul dengan lima baris per halaman.

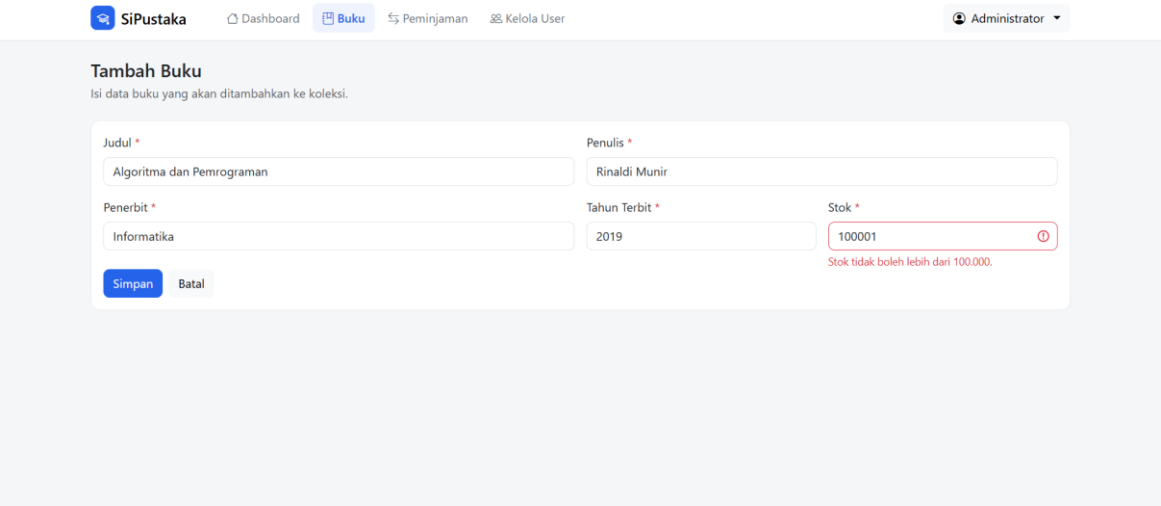
Gambar 8. Daftar buku yang dilihat user, tanpa kolom aksi



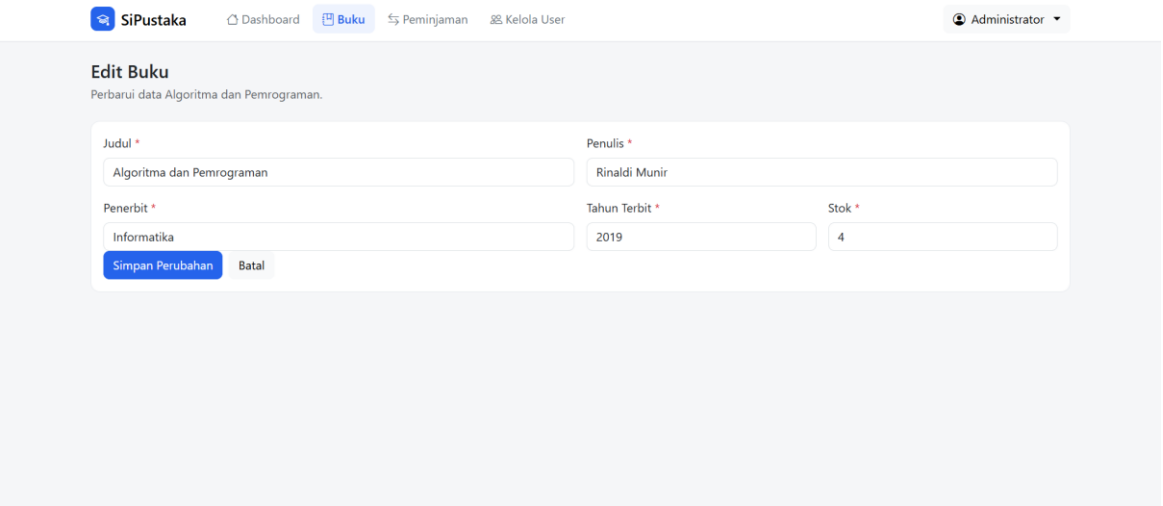
Gambar 9. Daftar buku pada akun admin beserta tombol aksi dan pagination



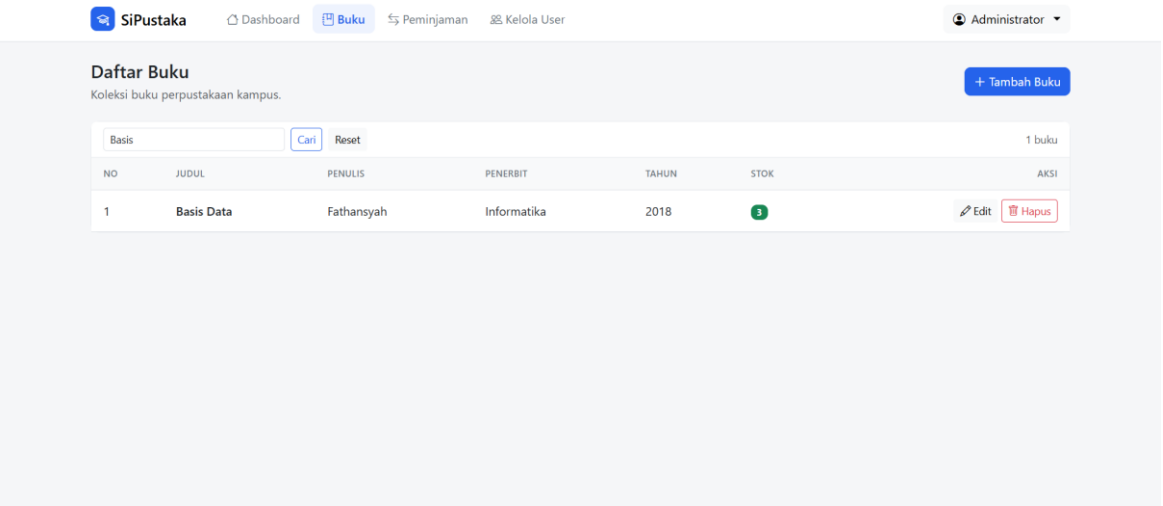
Gambar 10. Form tambah buku



Gambar 11. Pesan validasi ketika stok melebihi batas

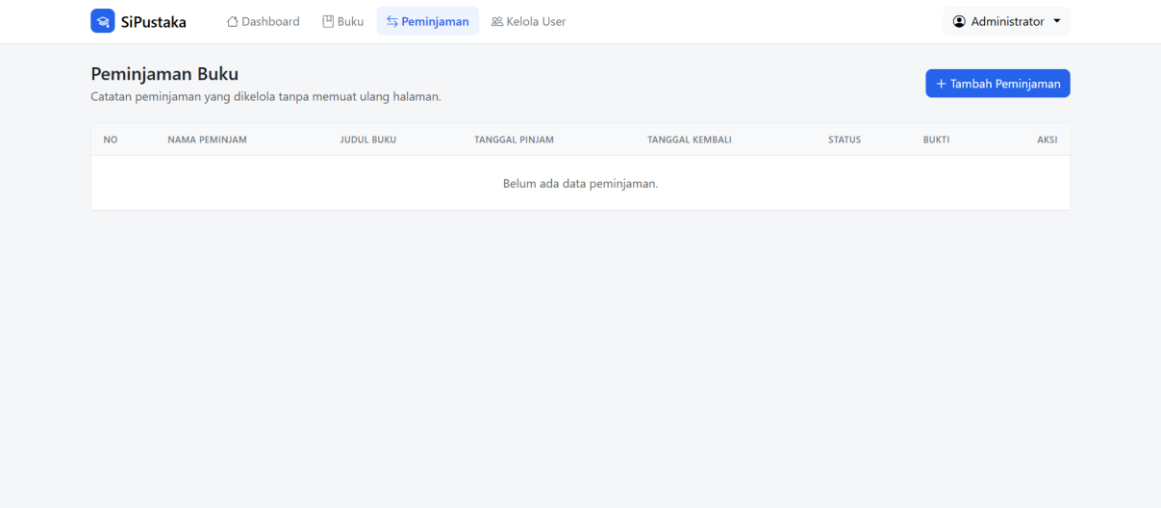


Gambar 12. Form edit buku

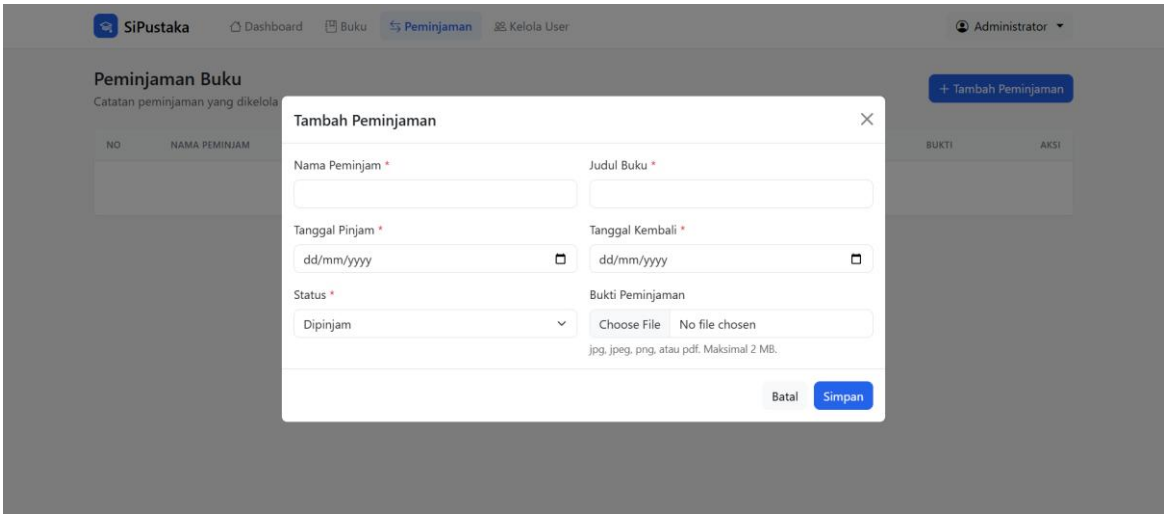


Gambar 13. Hasil pencarian kata kunci Basis

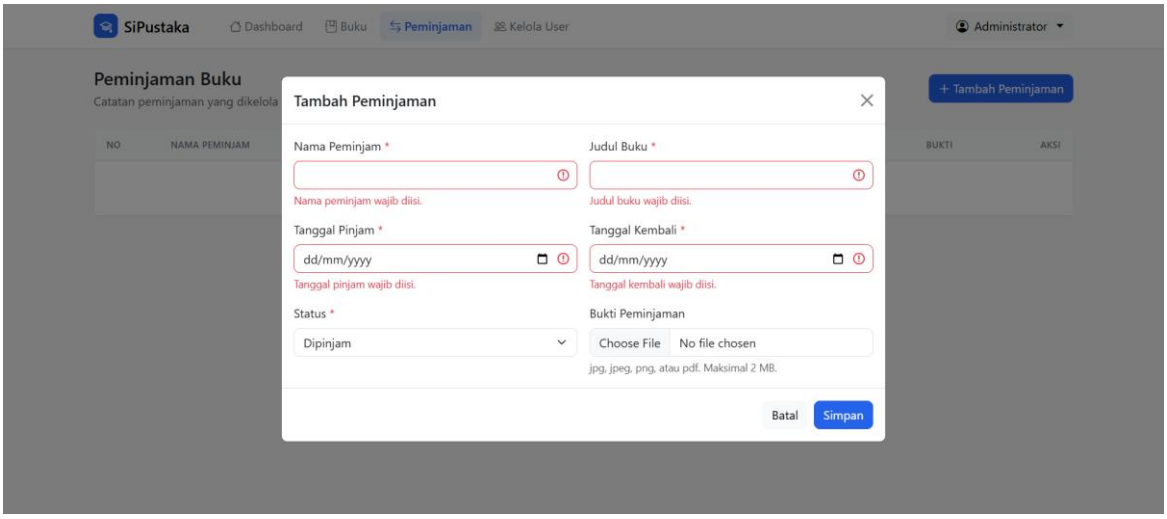
Modul Peminjaman diuji dengan menambah, mengubah, dan menghapus data tanpa memuat ulang halaman. Modal tambah memakai atribut novalidate supaya validasi dijalankan server dan pesan 422 dapat ditampilkan di bawah input.



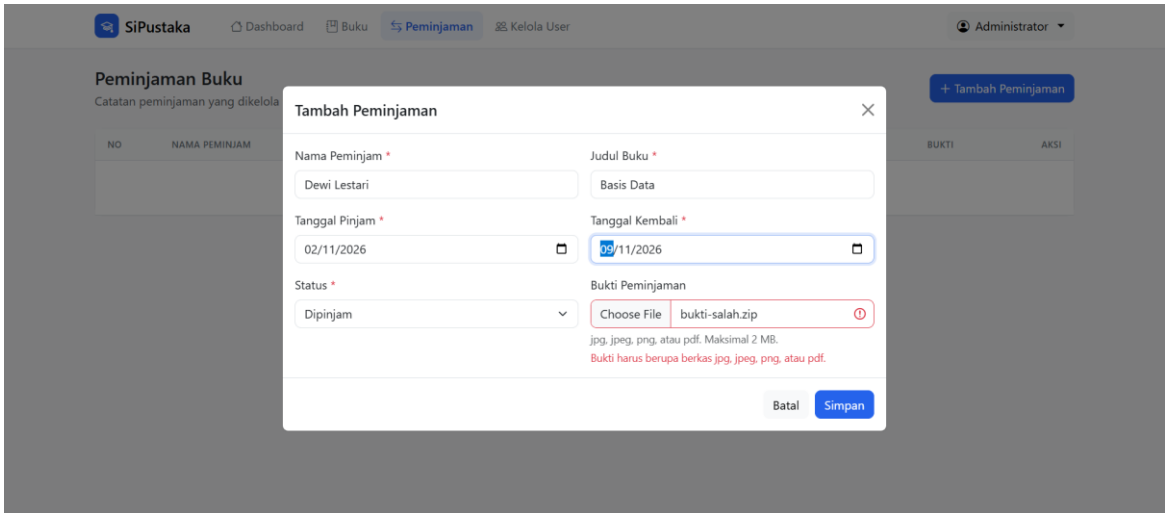
Gambar 14. Halaman peminjaman sebelum ada data



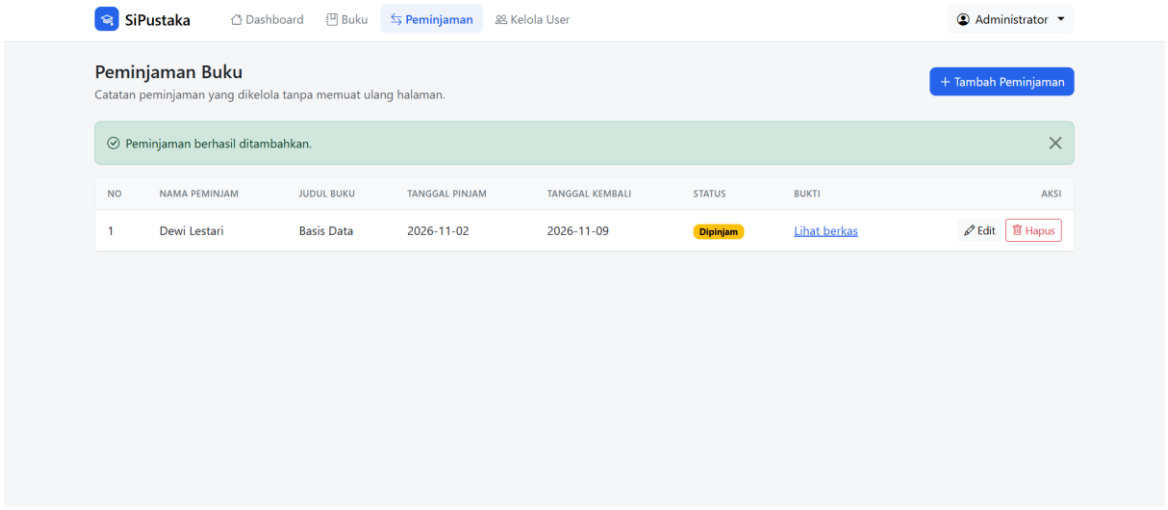
Gambar 15. Modal tambah peminjaman



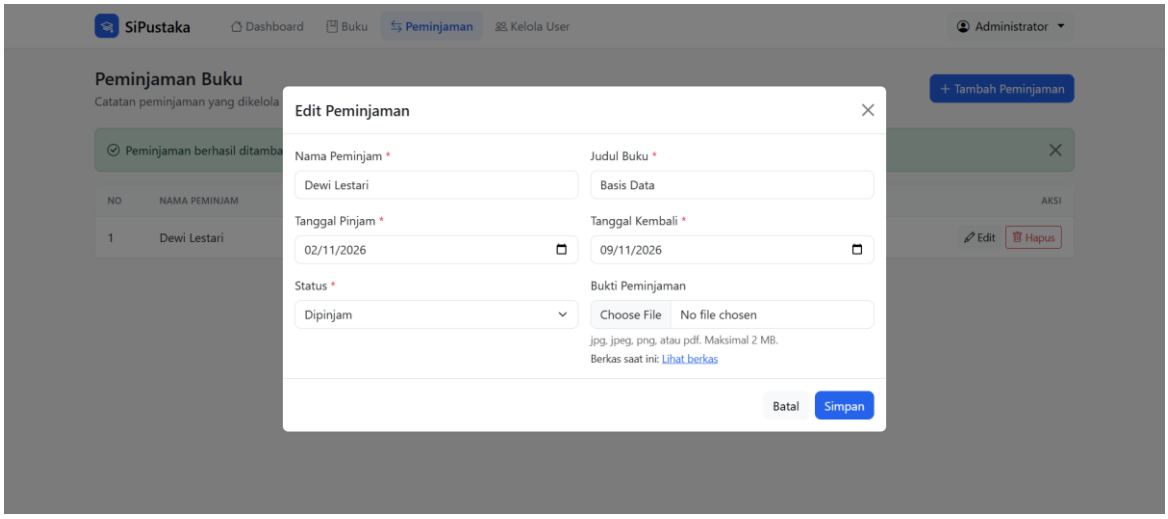
Gambar 16. Pesan error 422 di bawah input ketika form dikirim kosong



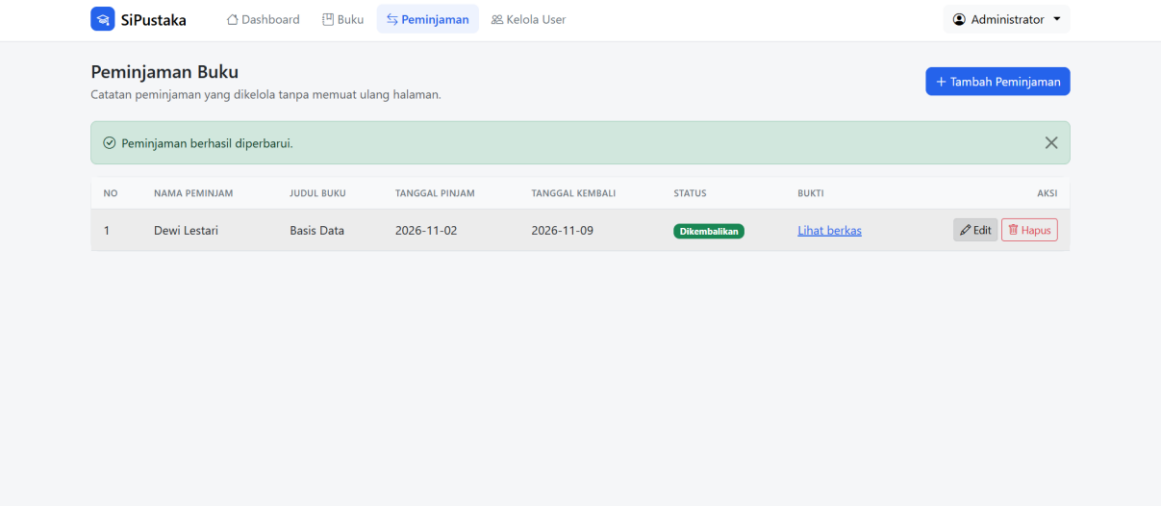
Gambar 17. Pesan error ketika berkas yang dipilih berformat ZIP



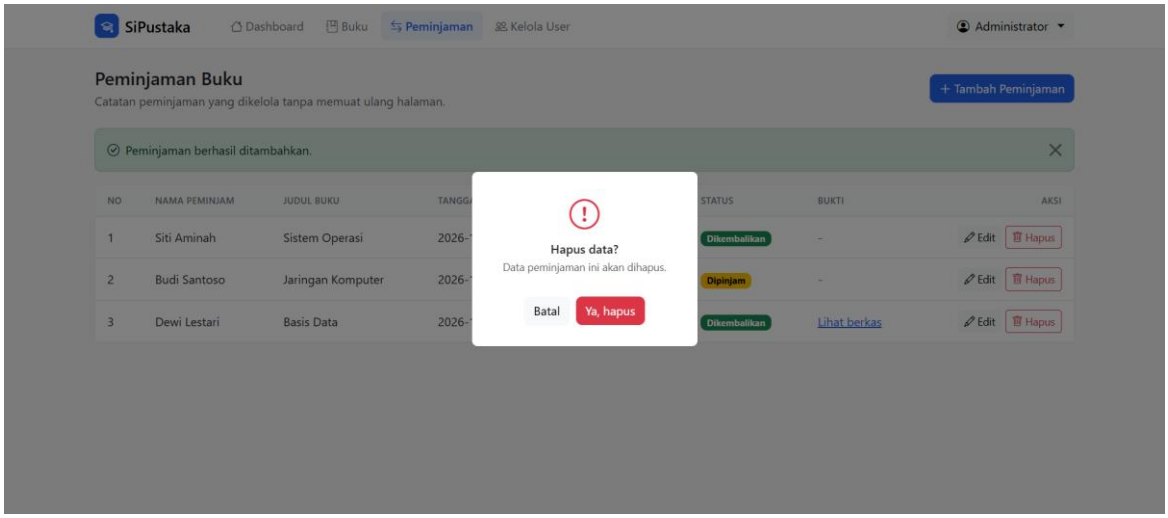
Gambar 18. Baris baru muncul di tabel setelah data disimpan, tanpa refresh



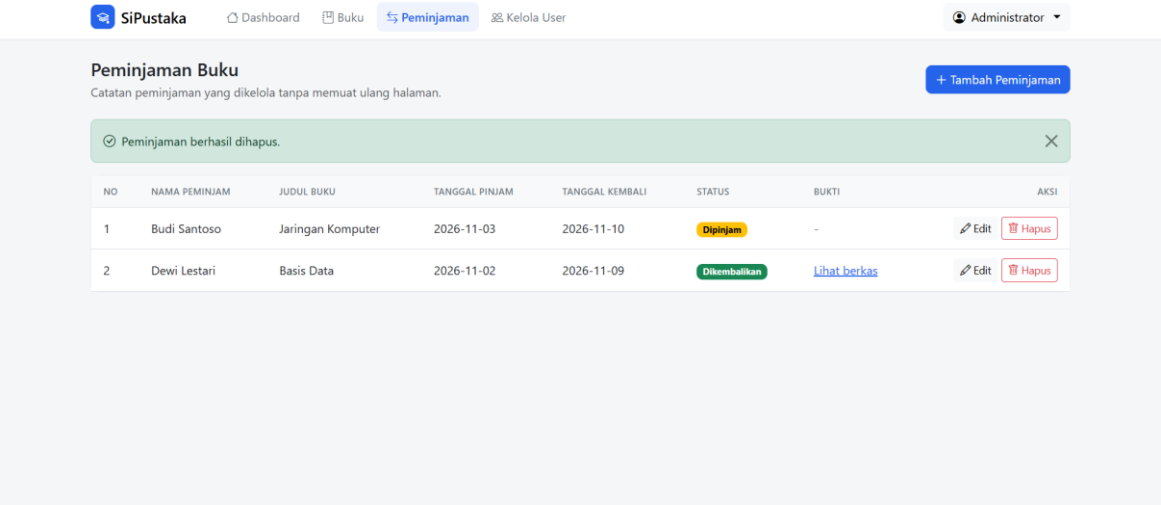
Gambar 19. Modal edit dengan tautan ke berkas bukti yang sudah tersimpan



Gambar 20. Status berubah menjadi Dikembalikan setelah proses edit

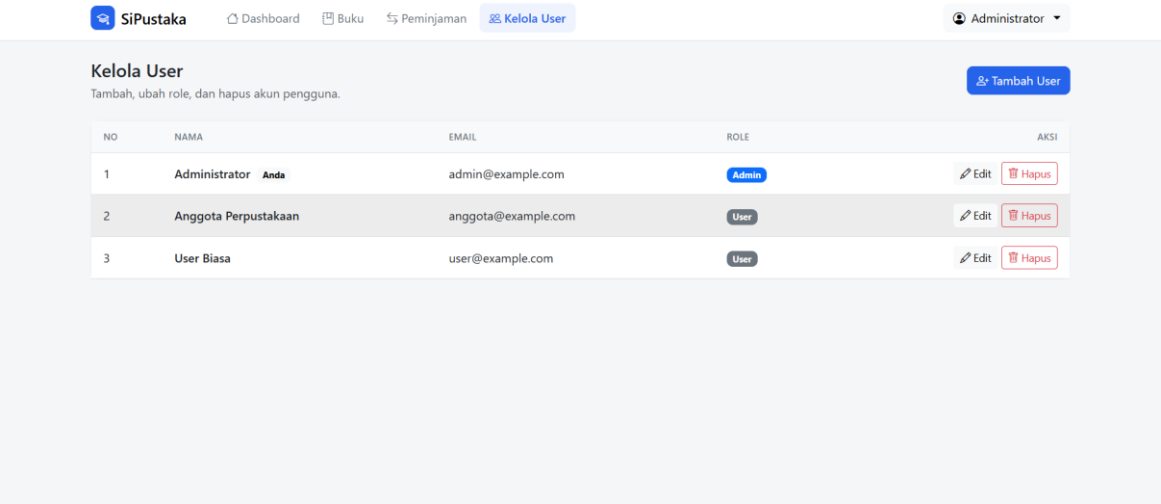


Gambar 21. Modal konfirmasi sebelum data dihapus

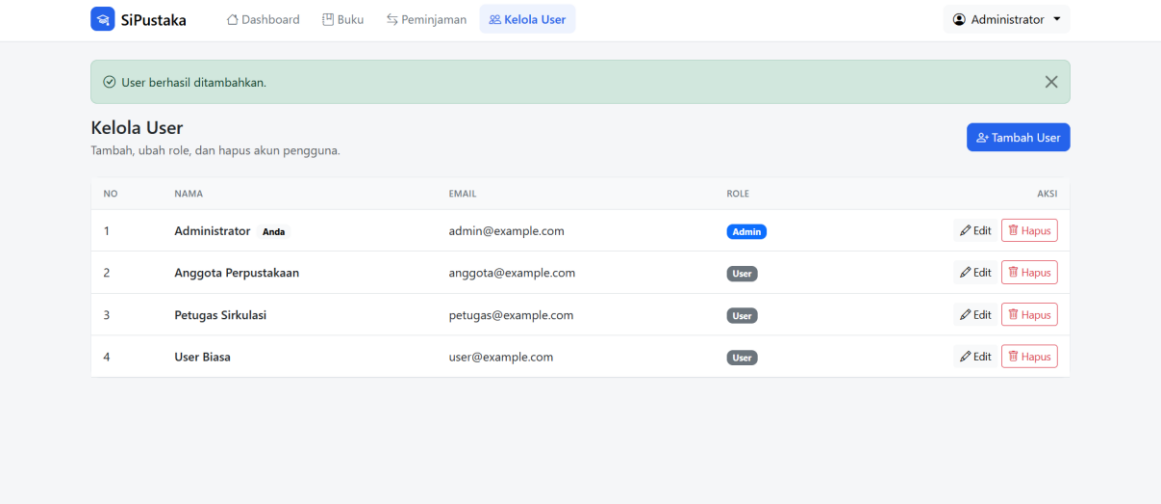


Gambar 22. Tabel setelah satu baris dihapus

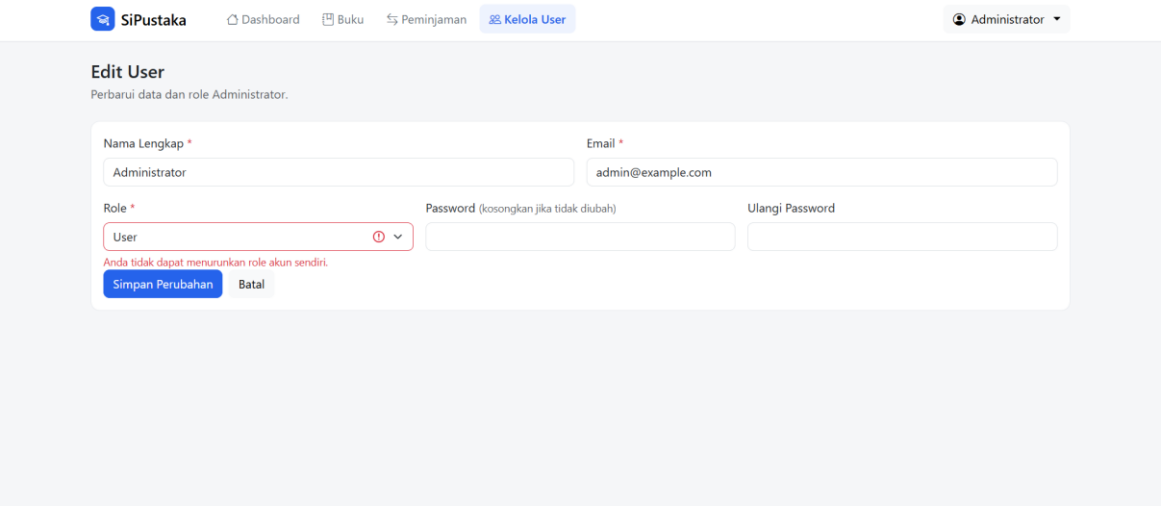
Modul UserManagement menampilkan daftar akun beserta badge role. Saya menguji penambahan akun, penolakan penurunan role akun sendiri, penolakan hapus akun sendiri, dan penghapusan akun lain.



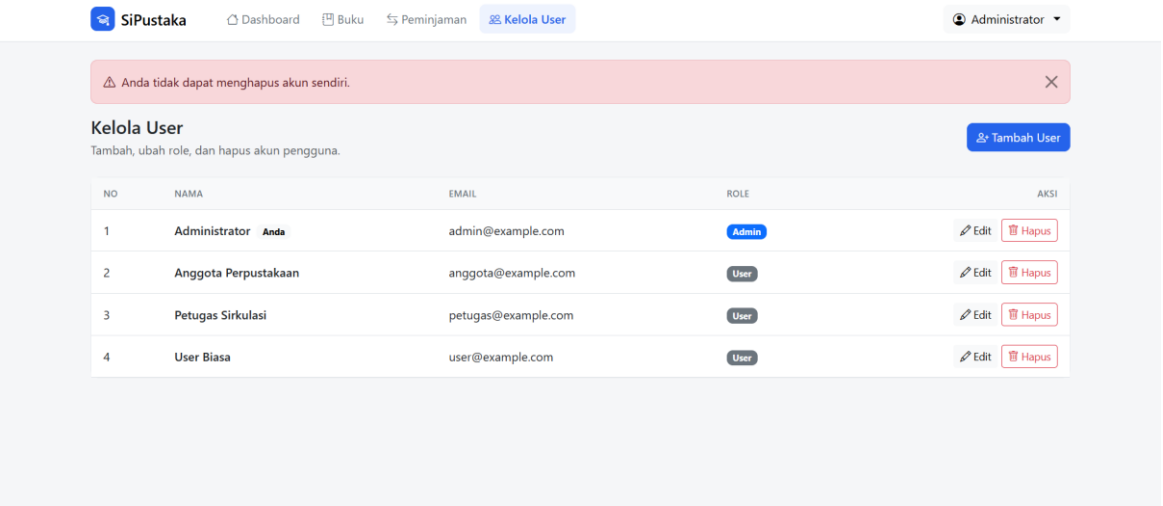
Gambar 23. Daftar user pada modul UserManagement



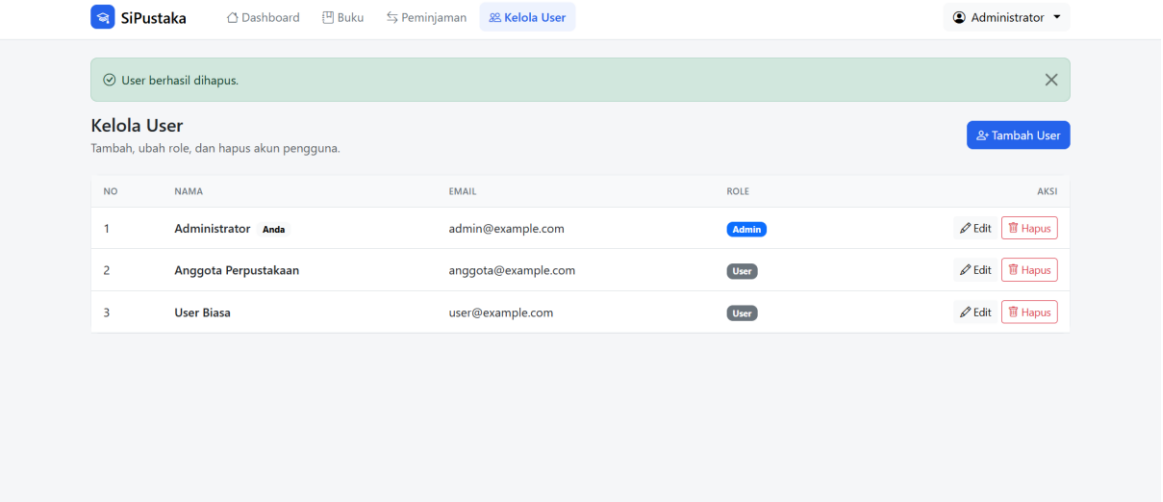
Gambar 24. Pesan berhasil setelah akun Petugas Sirkulasi ditambahkan



Gambar 25. Pesan error ketika admin mencoba menurunkan role akunnya sendiri



Gambar 26. Pesan error ketika admin mencoba menghapus akunnya sendiri



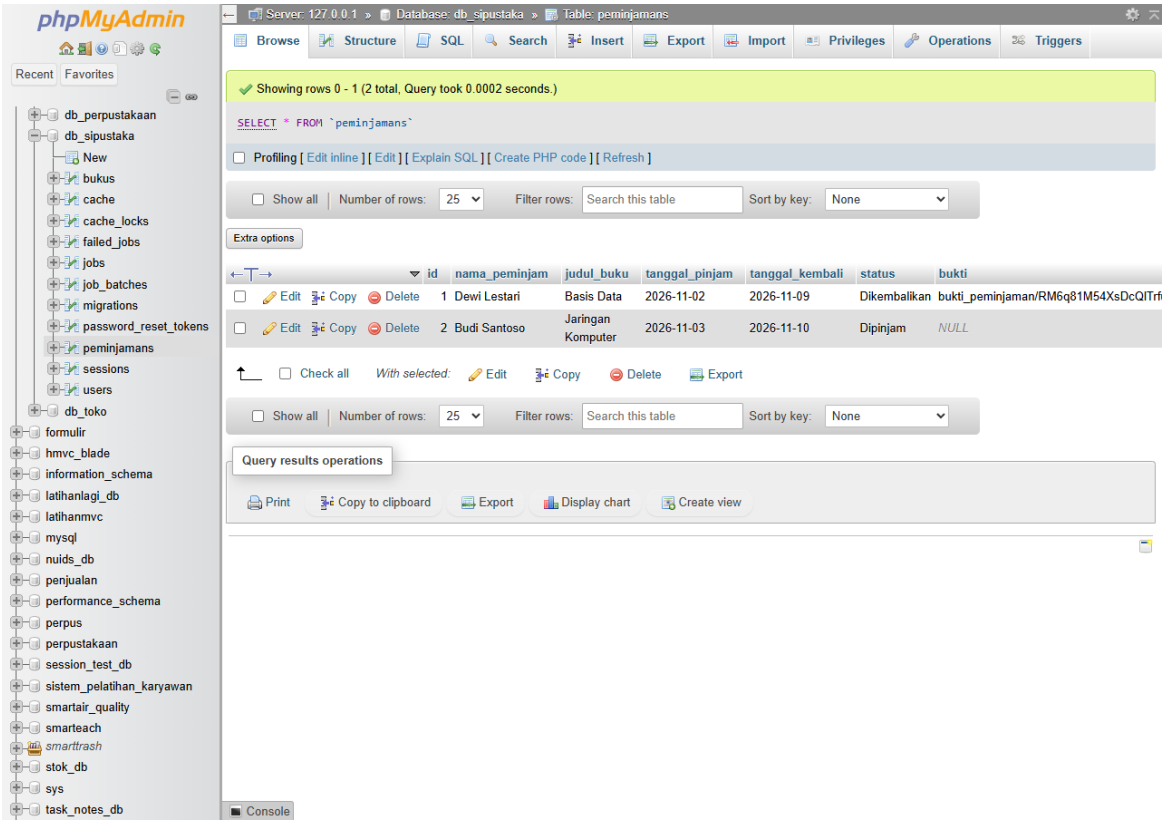
Gambar 27. Pesan berhasil setelah akun lain dihapus

Bukti bahwa modul Peminjaman memakai AJAX saya ambil dari respons JSON endpoint peminjamans/data. Respons yang sama dipakai jQuery untuk menyusun baris tabel tanpa memuat ulang halaman.

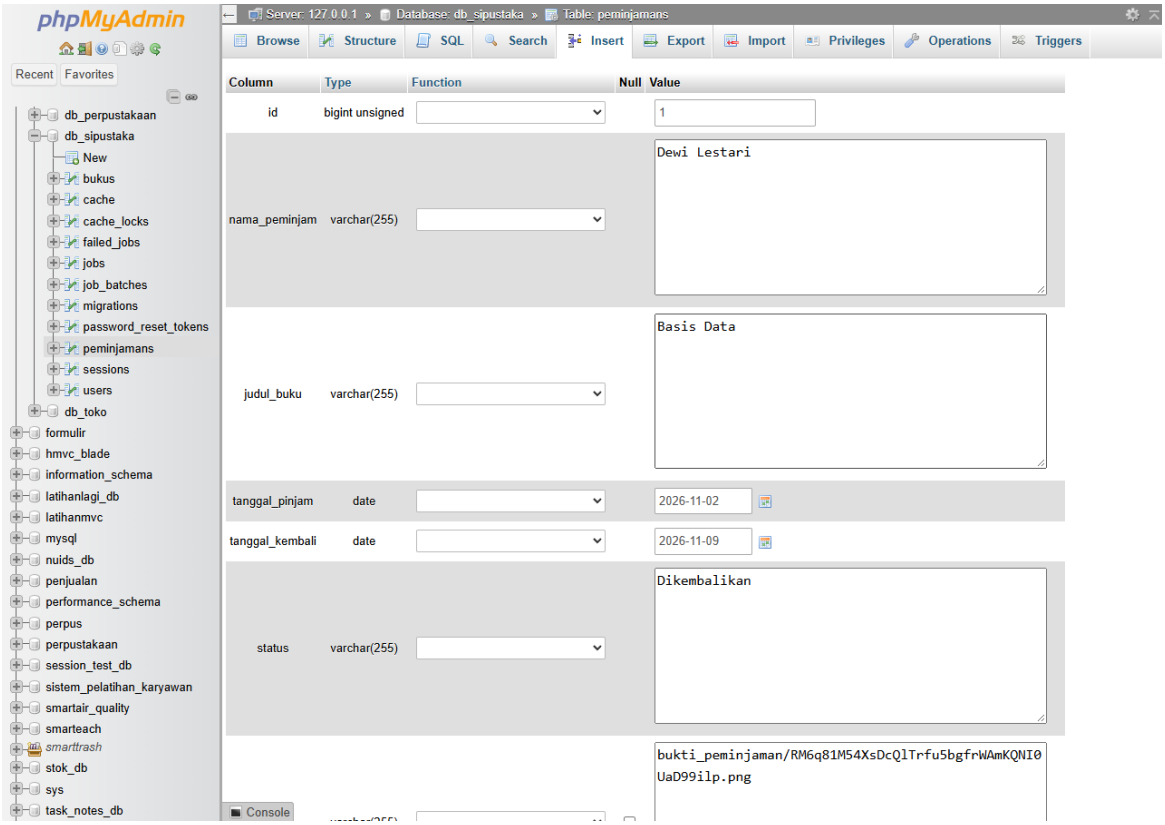


Gambar 28. Respons JSON dari endpoint peminjamans/data

Berkas bukti tersimpan di storage/app/public/bukti_peminjaman dan hanya path-nya yang masuk ke database. Tautan Lihat berkas bekerja setelah php artisan storage:link dijalankan. Saya memeriksa isi folder tersebut dan kolom bukti pada phpMyAdmin.



Gambar 29. Tabel peminjamans pada phpMyAdmin



Gambar 30. Nilai kolom bukti yang menunjuk ke folder bukti_peminjaman



Gambar 31. Isi folder storage/app/public/bukti_peminjaman

5. Analisis dan Pembahasan

Pemisahan kode ke dalam modul membuat route, controller, request, model, migration, dan view berada di satu folder. Modul Buku dan Peminjaman tidak perlu didaftarkan pada routes/web.php karena masing-masing modul memuat route-nya sendiri melalui service provider. Satu hal yang harus diingat, file composer.json setiap modul hanya terbaca setelah bagian merge-plugin ditambahkan. Tanpa bagian itu, autoload gagal menemukan class service provider modul, seperti yang terjadi pada percobaan pertama saya.

Authentication dan authorization dipisahkan dengan jelas. Login memeriksa email dan password, sedangkan hak akses ditentukan dari kolom role. Middleware role bekerja pada level route, sehingga user tidak dapat membuka halaman admin walaupun alamatnya diketik langsung. Menu Kelola User yang disembunyikan pada navbar hanya pengaturan tampilan, bukan perlindungan utama. Perlindungan sebenarnya ada pada middleware dan pada method authorize() di Form Request.

Perbedaan CRUD synchronous dan AJAX terlihat pada jumlah halaman yang dimuat. Pada modul Buku, setiap simpan dan hapus mengembalikan redirect sehingga halaman dimuat ulang dan pesan sukses dibaca dari session. Pada modul Peminjaman, controller mengembalikan JSON, lalu JavaScript memperbarui tabel. Pesan sukses ditampilkan oleh JavaScript, bukan dari session. Konsekuensinya, penanganan error juga berbeda. Form biasa memakai redirect dengan @error, sedangkan form AJAX membaca xhr.responseJSON.errors ketika status respons 422.

Pengiriman berkas pada AJAX memerlukan FormData. Jika form di-serialize, isi input file tidak ikut terkirim. Opsi processData: false mencegah jQuery mengubah FormData menjadi string, dan contentType: false membuat browser menulis sendiri boundary multipart. Di sisi server, berkas disimpan dengan store('bukti_peminjaman', 'public') sehingga nama file dibuat acak dan path-nya disimpan di kolom bukti. Pola ini membuat berkas tidak dapat ditebak dari nama aslinya, tetapi tetap dapat dibuka melalui symbolic link public/storage.

Ada dua kendala yang benar-benar muncul selama pengerjaan dan sudah saya perbaiki. Pertama, autoload modul gagal karena merge-plugin belum dikonfigurasi. Kedua, Eloquent membentuk nama tabel peminjamens dari nama model Peminjaman sehingga dashboard error 500. Keduanya tercatat pada storage/logs/laravel.log. Selain itu, pesan validasi bawaan Laravel untuk locale id tidak tersedia di project ini, jadi beberapa aturan saya beri pesan sendiri dan locale fallback diarahkan ke en. Teks pagination masih memakai bahasa Inggris dari Laravel.

Tangkapan tab Network pada DevTools tidak dapat saya sertakan karena pengujian dijalankan pada browser otomatis yang tidak menampilkan panel DevTools. Sebagai gantinya, saya membuka endpoint peminjamans/data dan menyertakan respons JSON-nya pada Gambar 28. Permintaan XHR yang sama juga terlihat pada kode JavaScript yang mengirim $.ajax dan memperbarui tabel tanpa reload.

6. Kesimpulan

Aplikasi SiPustaka berhasil dibangun dengan tiga modul HMVC. Modul Buku menangani CRUD synchronous dengan pencarian dan pagination, modul Peminjaman menangani CRUD AJAX beserta unggah bukti, dan modul UserManagement mengelola akun pengguna. Authentication ditulis manual di luar modul, sedangkan pembatasan hak akses memakai middleware role yang membedakan admin dan user.

Middleware role berhasil menolak user yang membuka halaman admin dengan kode 403, dan aturan pada modul UserManagement mencegah admin menghapus atau menurunkan role akunnya sendiri. CRUD AJAX berjalan tanpa memuat ulang halaman, termasuk saat mengirim berkas bukti dan menampilkan pesan error 422 di bawah input.

Dari pengerjaan ini saya memahami bahwa nama tabel perlu ditulis eksplisit ketika nama model berasal dari kata Indonesia, dan konfigurasi merge-plugin harus ada sebelum perintah dump-autoload dijalankan pada project yang memakai modul.

7. Link Repository Github

https://github.com/dhodhoo/pabw-andhika-yuridho